Security (System) Engineering with Generative AI

Homework Assignment no. 6

Student name: Brehon Parker

FAU Email: bparker2012@fau.edu

Algorithm:

I. Edit above your student name and homework number.
II. Edit below the missing information between < and >.
III. During development, for every significant task given to the agent you must:
   A. Take a screenshot of your VSCode window clearly showing the prompt you gave to the agent and the response it gave you back. Your FAU ID is the email address part excluding @fau.edu and it must be visible on the screenshot.
   B. Take a screenshot of your VSCode window clearly showing at least the top few lines of code that were generated or modified by the agent. Your FAU ID must be visible on the screenshot.
   C. Insert the screenshots into the homework Word document, along with the task prompt given to the agent pasted as regular text.
IV. When finished, convert the Word document to PDF and upload on the homework Canvas page.

Screencast Recording URL:

https://www.loom.com/share/c0d303d6cde44483b8674fa29f8042da

Github Reposity URL:

https://github.com/bparker2012-cyber/gensec-code/tree/codex/hw6-secure-notes/hw6

Application name: *Secure Notes*

Application Description: *Secure Notes is a local Flask application generated with OpenAI Codex. Separate user accounts can create, read, edit, and delete persistent private notes stored in SQLite.*

Development task and coding agent

Coding agent: OpenAI Codex, Codex desktop app. I accepted the secure notes proposal and requested methodical GitHub commits. The following task text and replies come from the actual Codex conversation export. Prepared replay prompts are identified separately.

Accepted application proposal

My recommendation is a secure notes application built with Codex: login, private notes, SQLite storage, password hashing, CSRF protection, and input validation. It connects naturally to Lab 6 and gives us clear security demonstrations that fit the recording limit.

Actual task prompt

lets get it donev:) remmeber github commits should be methodiically as we would typically do....

Actual Codex response

I’ll build the secure notes app, save the prompts and evidence, and prepare the submission documents and recording script. I’ll commit in clear stages: scaffold, working app, security checks, then submission materials.

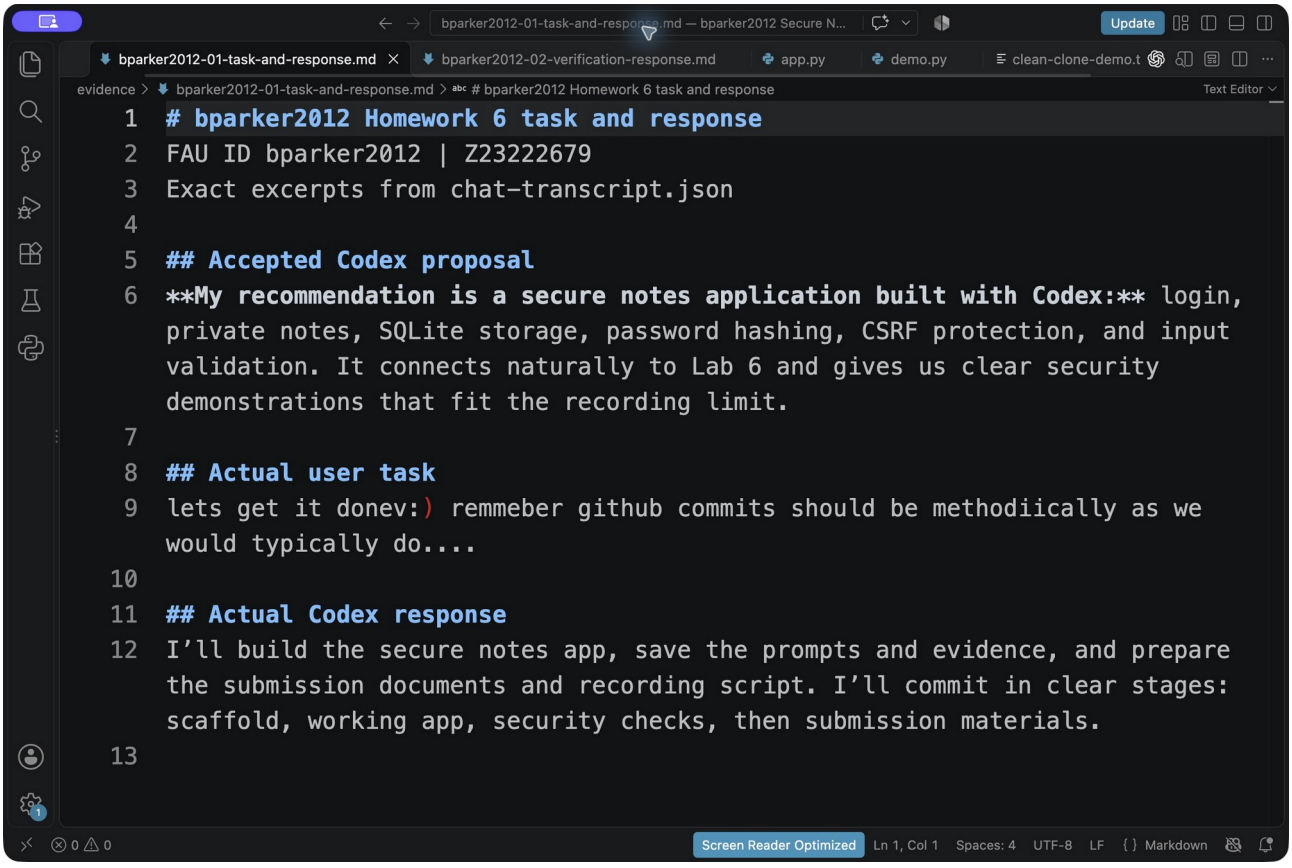


Figure 1. VS Code displays exact excerpts of the real task and Codex response. The file identifies its conversation-export source and the FAU email ID is visible.

Standalone prompts for recreation

These self-contained instructions were prepared from the agreed task for the required screencast replay. They are not additional historical user messages. The complete wording and provenance are preserved in prompts.txt.

Prompt 1 Build the application

Build a small Python 3.13 Flask application named Secure Notes inside hw6. Use app.py with a create_app factory, Jinja templates, local CSS, and SQLite. Implement registration, login, logout, and create/read/edit/delete private notes. Use a clean, readable interface with a sidebar, note list, editor, clear errors, and a footer showing Brehon Parker, bparker2012@fau.edu, and Z23222679. Keep the app usable without external APIs, JavaScript, or internet services. Treat every form field, URL, and cookie as adversarial. Use Werkzeug scrypt password hashing, parameterized SQL, owner-scoped queries on every note route, Jinja autoescaping, per-session CSRF tokens on every POST including auth and logout, session clearing at auth transitions, bounded input sizes, a persistent random signing key outside Git, HttpOnly/SameSite cookies, no-cache responses, security headers, trusted loopback hosts, and debug mode off. Add a bounded SQLite-backed login/registration rate limit. Store only an opaque session token in the signed cookie and its hash server-side so logout revokes captured cookies. Run on 127.0.0.1:5006 by default, with optional Secure cookies for HTTPS. Document that note content is plaintext at rest, local HTTP is a classroom mode, and this is not a production deployment. Pin dependencies with pyproject.toml and uv.lock, and ignore databases, signing keys, virtual environments, and caches. Keep the scope small enough to explain in a five-minute screencast.

Prompt 2 Verify security and prepare the demonstration

Exercise the real application and add meaningful automated tests for private note isolation, anonymous access, CSRF rejection, stored XSS escaping, SQL injection attempts, password hashing, revoked sessions, malformed and oversized input, security headers, and rate limiting. Fix any failures without weakening the controls. Add a reproducible --demo security-check command using temporary data and Flask's test client, clearly labeled as in-process checks. Do not add an HTTP test endpoint. Supply setup/run/test instructions, a security overview with honest limitations, and a short recording script that follows the required order: git clone, prompt walkthrough, live regeneration in a fresh directory, security explanation, functionality and limitations. Preserve real evidence; never fabricate a model response, screenshot, recording URL, or submission.

Recorded replay request

The recorded Codex session read the two prompts above and regenerated the application. The following VS Code views show the exact additional request, including the fixed package versions, executable commands, and instruction to verify the result. The full visible conversation export is retained in evidence/replay-conversation.json.

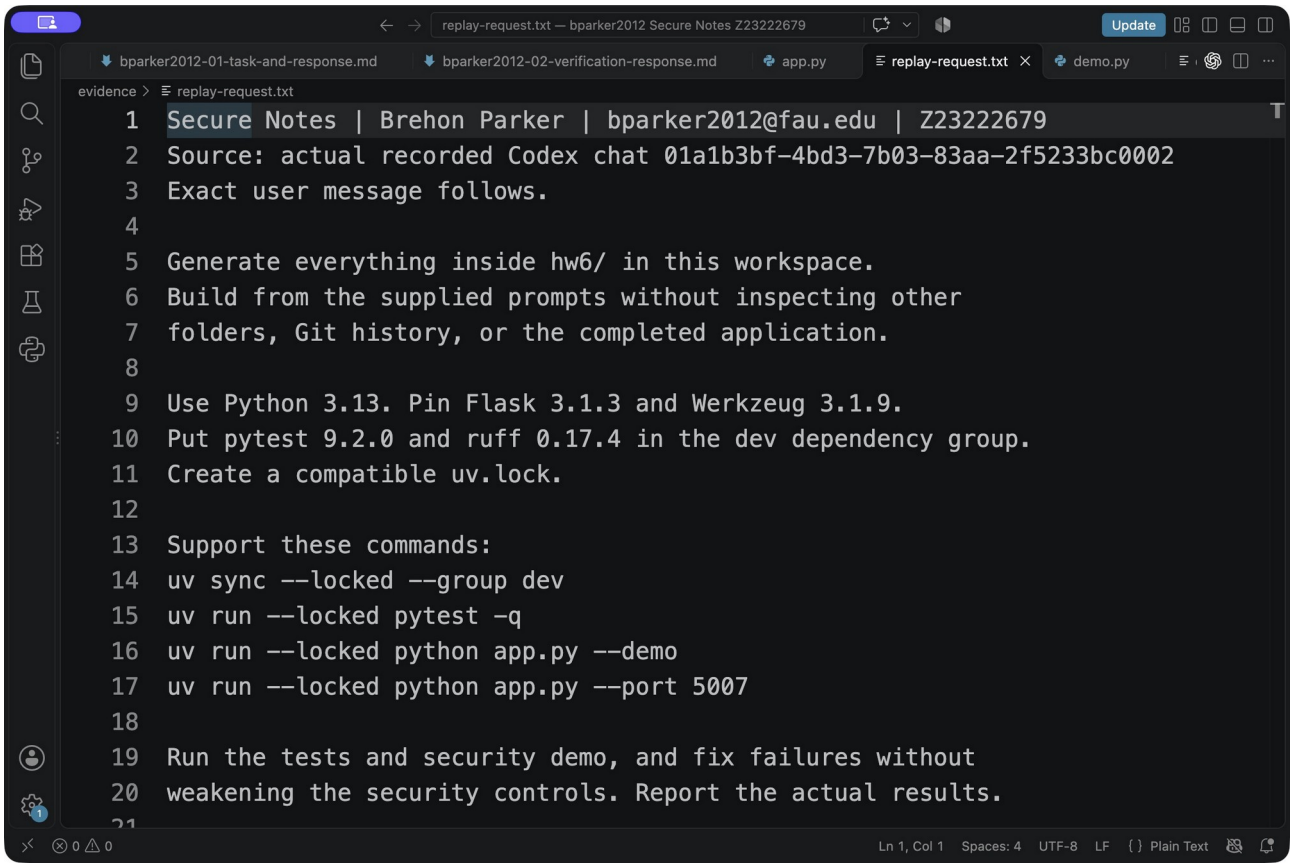


Figure 2. Exact request from the recorded regeneration, displayed in VS Code with the FAU email visible.

I'll check for the supplied prompts at the workspace root and in `hw6/`, then build and verify everything inside `hw6/`. I won't inspect other folders or Git history, start the server, or commit anything.

Recorded replay request continued

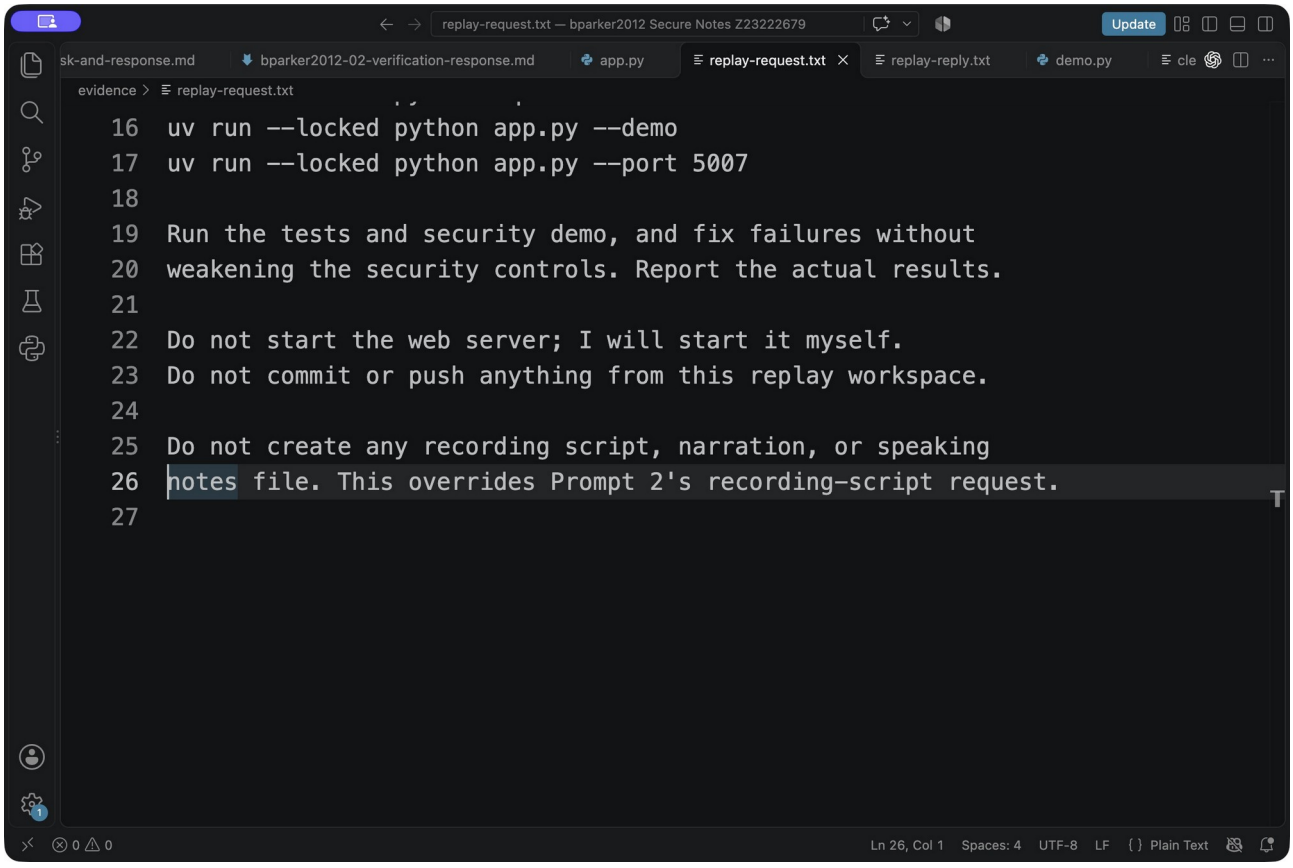


Figure 3. Remaining lines of the actual replay request. This is the continuation of Figure 2.

The separate Codex chat worked in the course repository. It read the saved prompts, removed the preceding application source, and generated new source and tests inside hw6. The original implementation remains available in earlier Git commits. The regenerated application is preserved in commit cbb91a3868a6.

The standalone prompt file and the actual replay request have different roles: the former specifies the application, while the latter directs the recorded regeneration and verification. Both are retained in prompts.txt.

Recorded replay request as text

Exact technical user message from the recorded regeneration, reproduced as ordinary selectable text in addition to Figures 2 and 3.

Generate everything inside hw6/ in this workspace.
Build from the supplied prompts without inspecting other
folders, Git history, or the completed application.

Use Python 3.13. Pin Flask 3.1.3 and Werkzeug 3.1.9.
Put pytest 9.2.0 and ruff 0.17.4 in the dev dependency group.
Create a compatible uv.lock.

Support these commands:
uv sync --locked --group dev
uv run --locked pytest -q
uv run --locked python app.py --demo
uv run --locked python app.py --port 5007

Run the tests and security demo, and fix failures without
weakening the security controls. Report the actual results.

Do not start the web server; I will start it myself.
Do not commit or push anything from this replay workspace.

Do not create any recording script, narration, or speaking
notes file. This overrides Prompt 2's recording-script request.

Generated application code

During the recorded replay, Codex generated app.py, local Jinja templates, CSS, a locked dependency environment, and security tests. The current source supports registration, note creation, reading, editing, deletion, and sign-out.

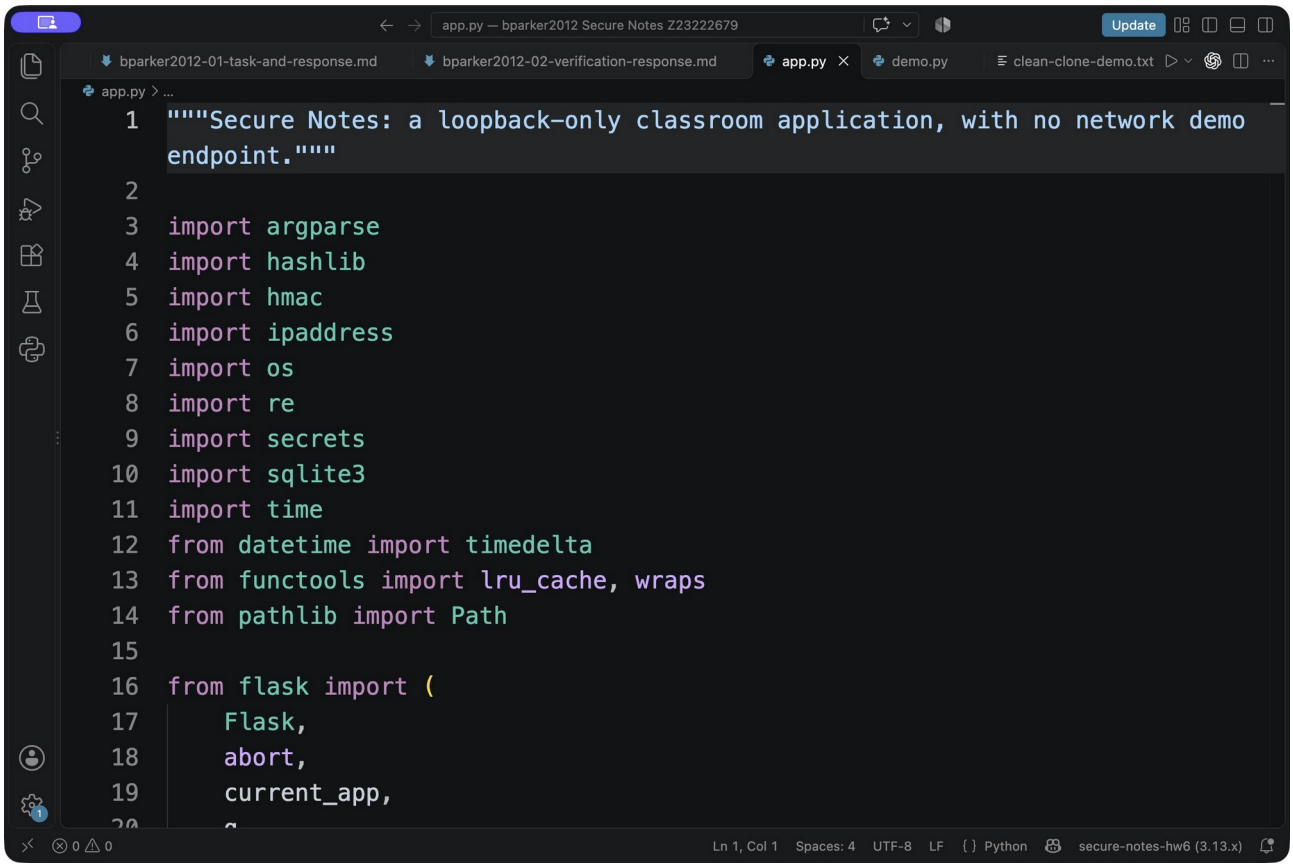


Figure 4. Top lines of the regenerated Python application in VS Code, with bparker2012 visible in the window title.

The source remains readable and divided into a small application factory, database helpers, authentication checks, and note routes. The UI requires no JavaScript or remote services.

Security verification and agent replies

The recorded session reported 120 passing tests and 13 passing security checks. Locked installation, lock validation, lint, and formatting also passed. The image below displays the exact final reply from the conversation export.

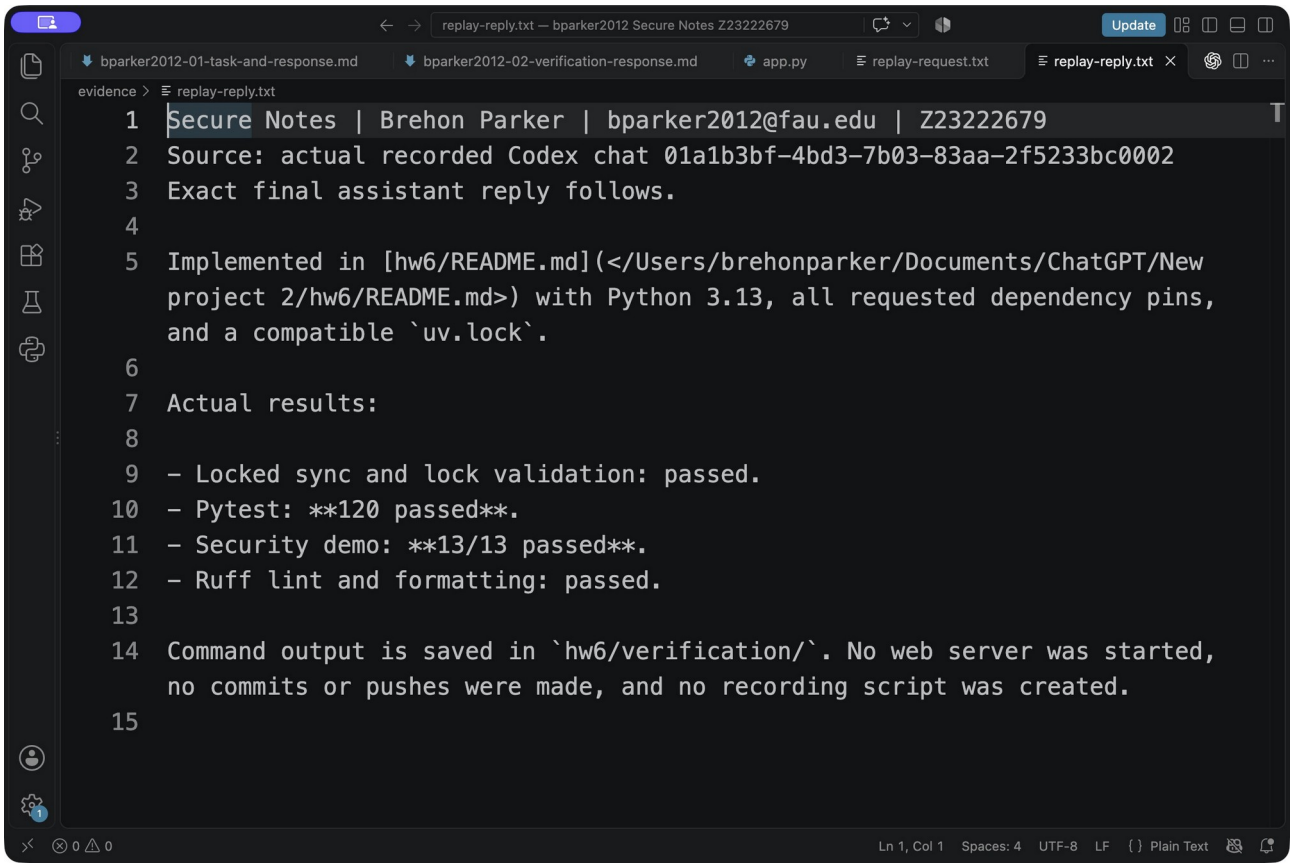


Figure 5. Exact final reply from the recorded Codex session, showing 120 passing tests and 13 passing security checks.

The suite covers unauthorized note access, missing or malformed CSRF tokens, stored XSS, SQL injection, password hashes, expired or revoked sessions, oversized inputs, host rejection, persistent signing keys, bounded state, and concurrent authentication attempts. The tests exercise real routes and SQLite.

Generated verification code

Codex added adversarial-input tests and a disposable demonstration. The --demo command calls actual Flask routes and SQLite through the test client. It uses temporary synthetic data and is separate from the live browser demonstration.

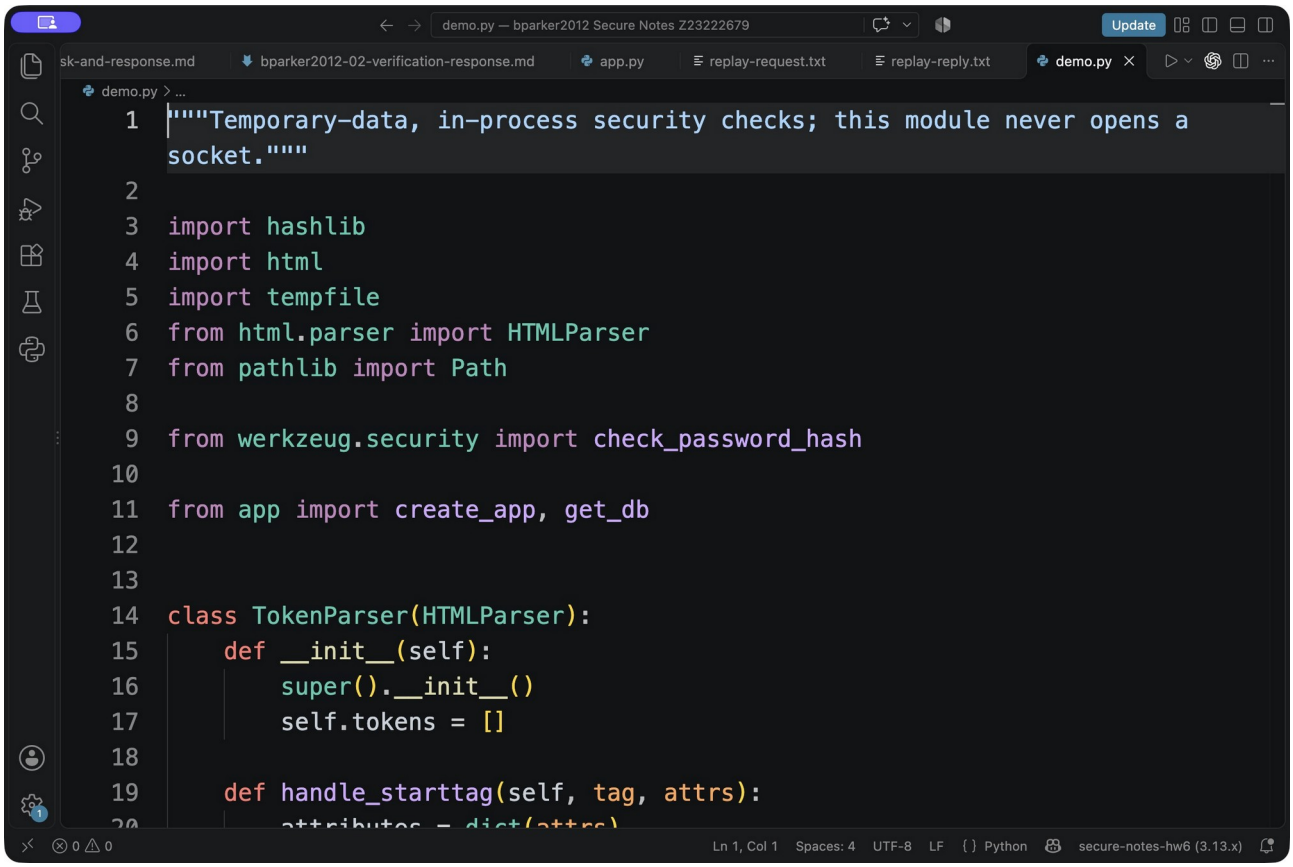


Figure 6. Top lines of the regenerated in-process demonstration code, with the FAU ID visible.

Private note ownership

Every note lookup includes both the note ID and the authenticated user ID. Update and delete statements repeat the owner condition. A second account cannot list, open, edit, or delete the first account's note; these requests are covered by the passing test suite.

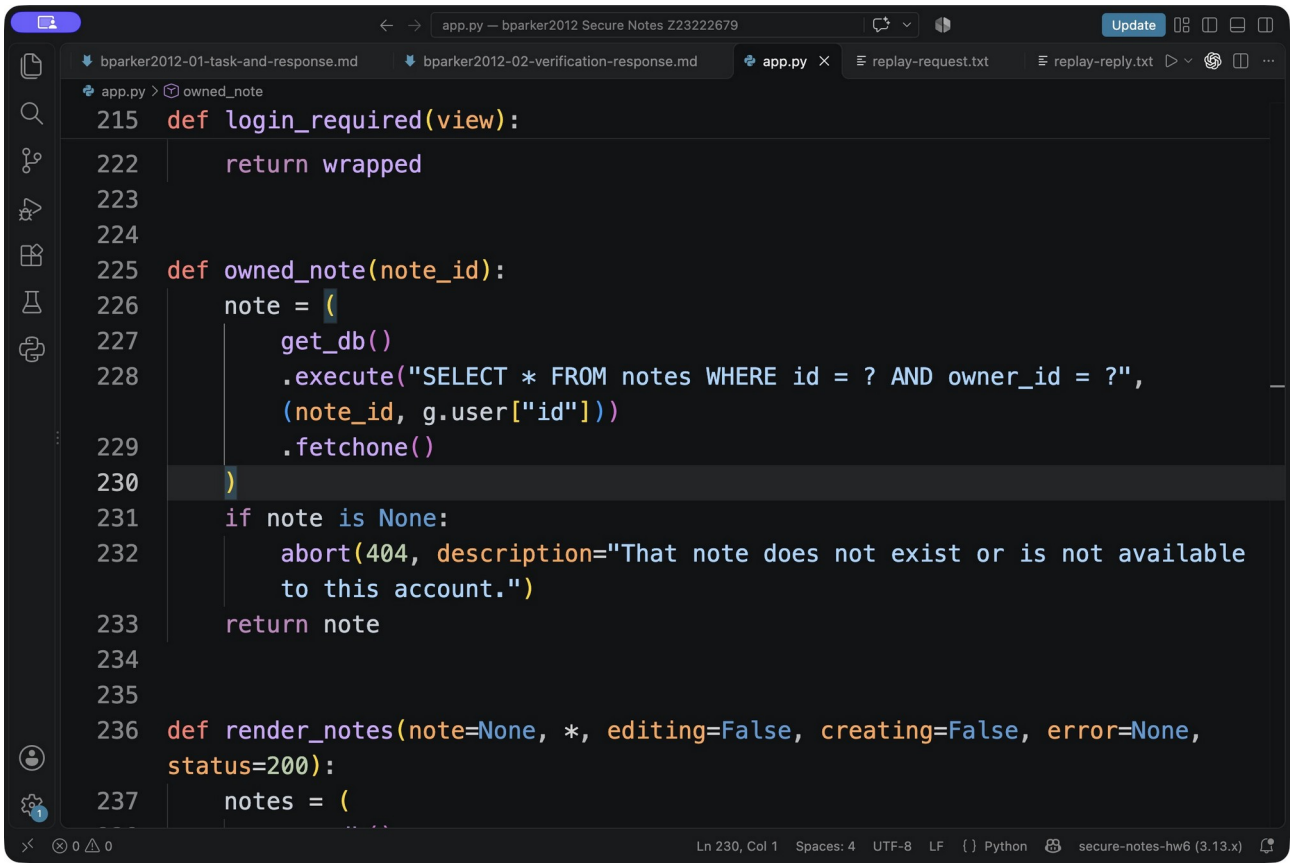


Figure 7. The regenerated query binds both the note ID and the authenticated owner ID.

CSRF and authentication sessions

Every POST, including login, registration, and logout, verifies the session CSRF token. Authentication transitions clear session state and rotate the token. The signed cookie contains only an opaque session identifier. SQLite stores its hash and the CSRF token, allowing logout and subsequent login to revoke captured old cookies. Sessions have a fixed one-hour expiry. Cookies are HttpOnly and SameSite Strict; Secure cookies can be enabled when actual HTTPS is provided.

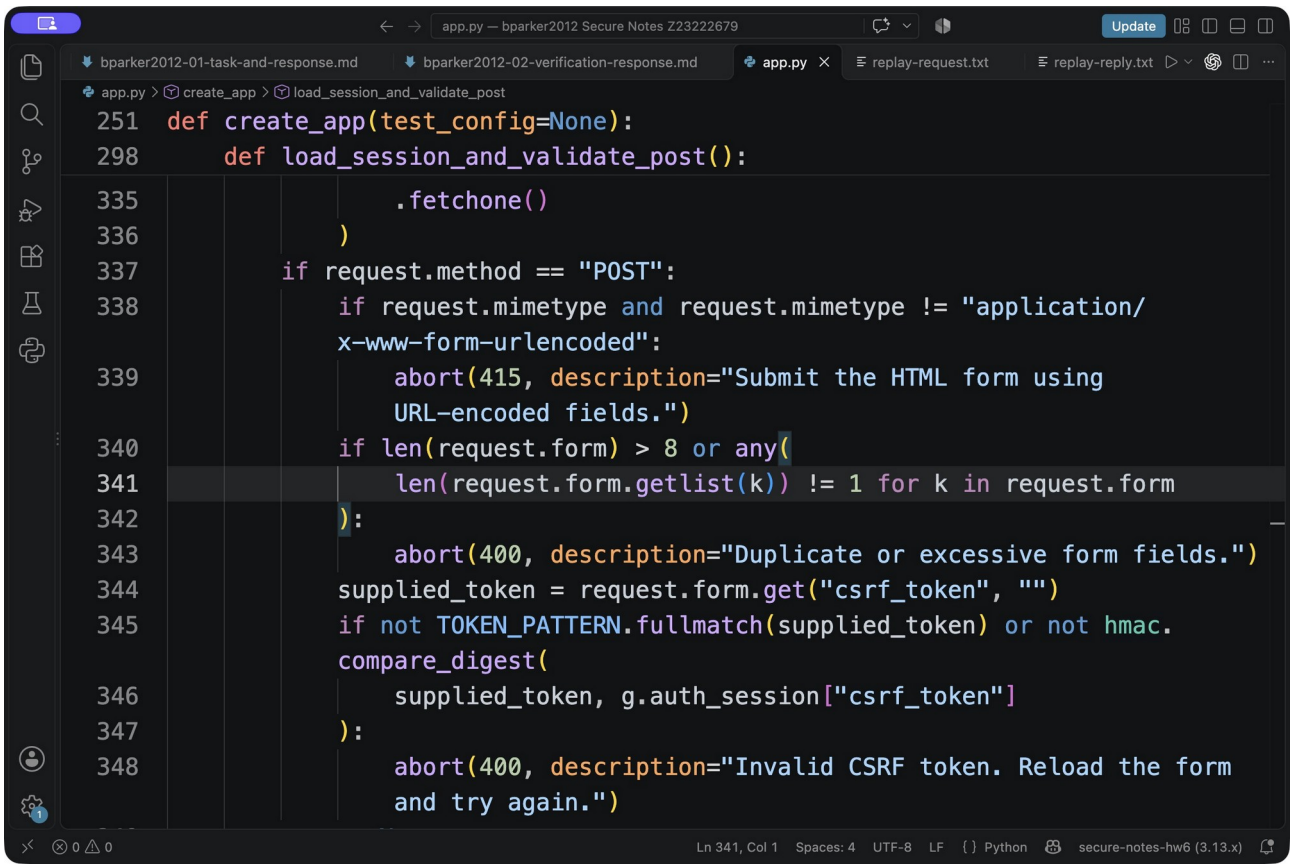


Figure 8. The regenerated request guard rejects duplicate fields and checks every POST token.

Verification from a fresh GitHub clone

A fresh clone of remote commit cbb91a3868a6 installed the locked dependencies, passed all 120 tests, passed all 13 security demonstration checks, and passed Ruff lint and formatting checks. This independently verifies the regenerated source from GitHub. Receipts are retained under verification/.

..... [60%]
..... [100%]
120 passed in 8.31s

Actual security demonstration output

```
Secure Notes — IN-PROCESS security checks (Flask test client)
Temporary SQLite data only. No server or network listener is started.
PASS 01: registration and authentication rotate session and CSRF tokens
PASS 02: anonymous users cannot read or create notes
PASS 03: owner checks isolate note lists, reads, edits, and deletes
PASS 04: missing and cross-session CSRF tokens cannot perform actions
PASS 05: stored XSS stays escaped in the reader and textarea
PASS 06: SQL injection strings remain data and cannot bypass authentication
PASS 07: passwords use salted Werkzeug scrypt hashes
PASS 08: signed cookies contain only an opaque token; SQLite stores its hash
PASS 09: malformed fields, oversized requests, and enormous note IDs fail safely
PASS 10: security headers, no-cache responses, and trusted hosts are enforced
PASS 11: logout revokes captured cookies, including mutation attempts
PASS 12: SQLite authentication rate limits persist across app restarts
PASS 13: the owner can log back in, read, edit, and delete private notes
RESULT: 13/13 in-process security checks passed; temporary data removed.
```

The original runtime dependency audit is retained under evidence/runs/ and covers the pinned runtime versions. The fresh-clone checks above establish the current test and demonstration results. These checks are not a complete security audit.

Browser demonstration

The regenerated application was also exercised in a browser using a separate temporary database and synthetic account. Registration and note creation succeeded. The reader shows HTML-like input literally, and exposes edit and delete controls. The full create/read/edit/delete and sign-out flow is covered by the passing tests and the student's recorded demonstration.

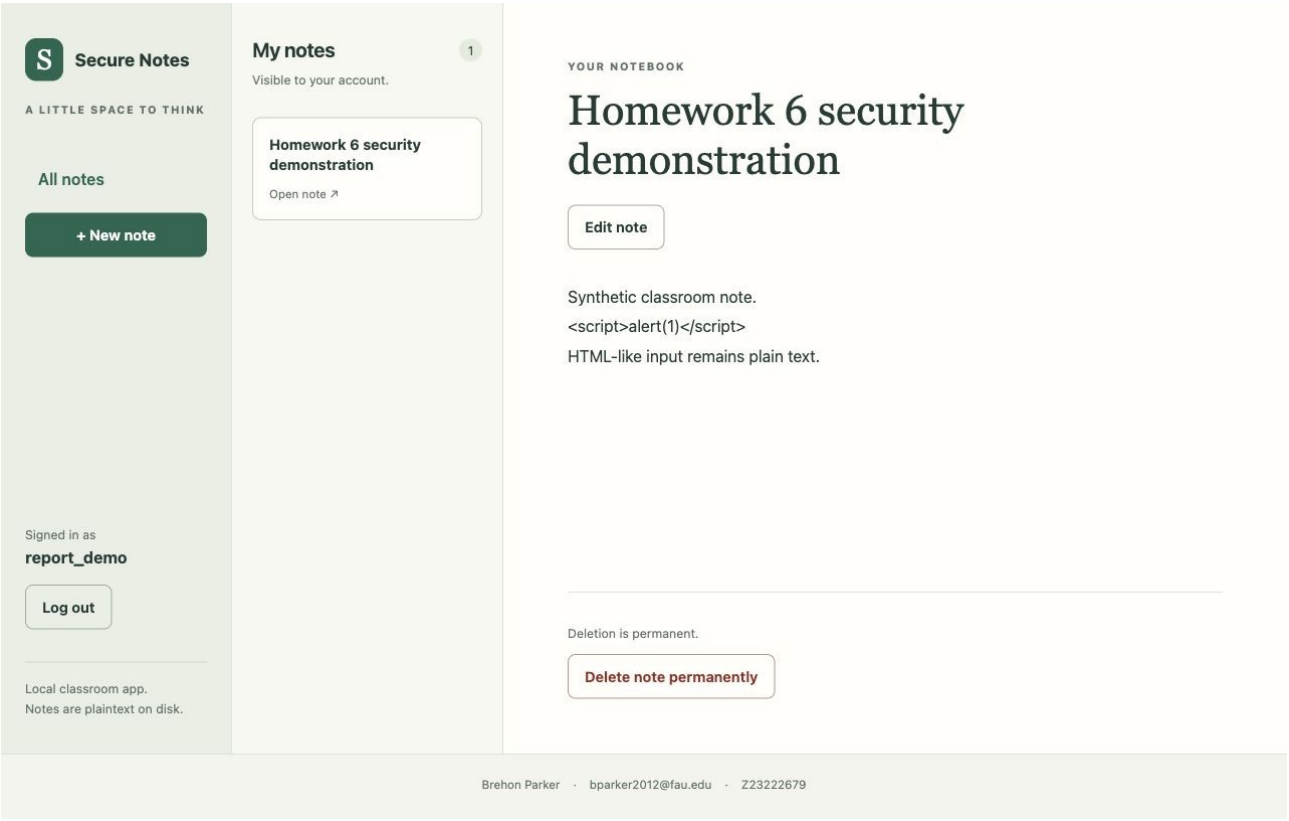


Figure 9. The regenerated notebook with synthetic note content, escaped script-like text, and the FAU email in the footer.

Capabilities and limitations

The app supports separate accounts and persistent private notes, with password hashing, parameterized SQL, output escaping, owner checks, CSRF verification, revocable sessions, request limits, and authentication rate limits. Note content is plaintext at rest. Filesystem access exposes it, and deleted data may remain in database pages or backups. The app has no MFA, password recovery, sharing, or attachments. Public deployment requires HTTPS, a production server, operational controls, and further security review.

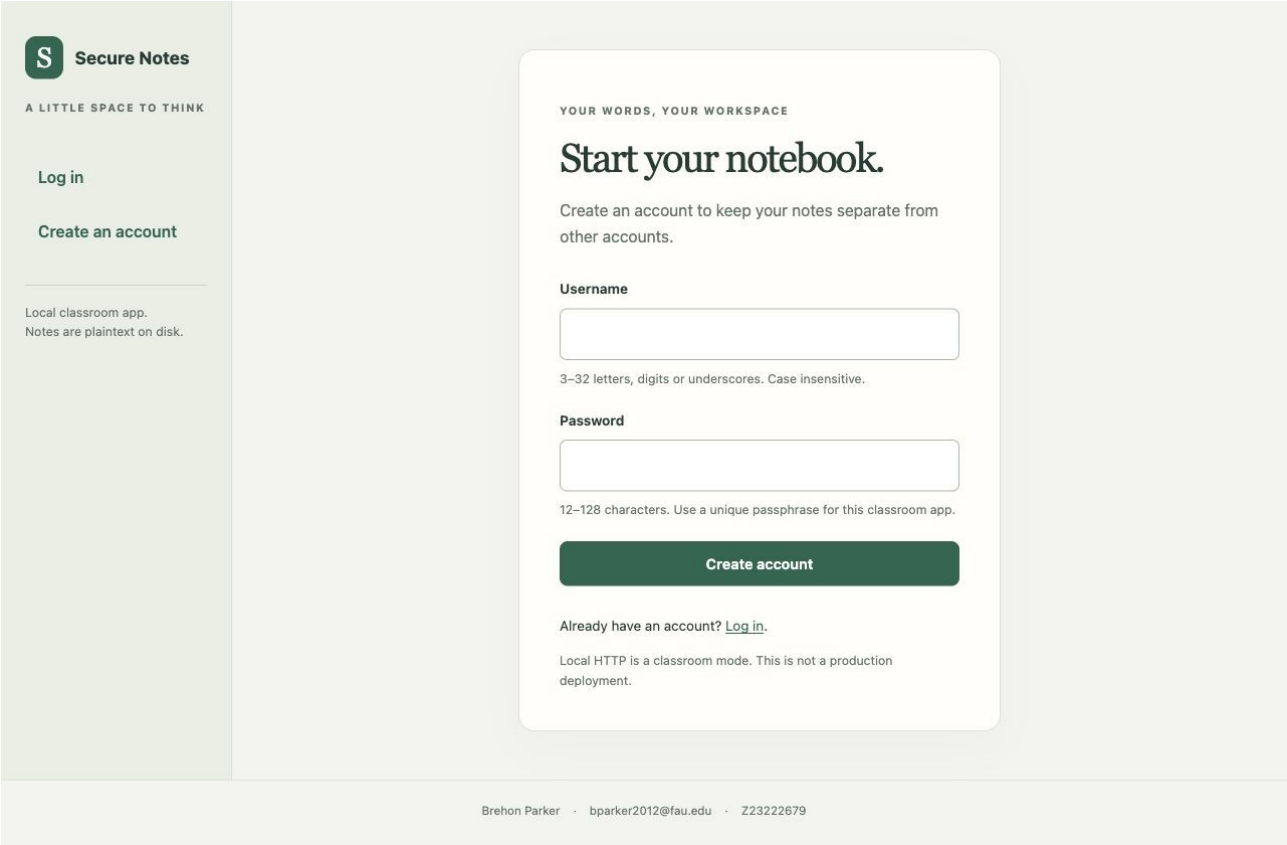


Figure 10. The regenerated registration page identifies the local classroom mode and plaintext storage limits.

Current bounds include 120-character titles, 20,000-character note bodies, 96 KiB requests, 1,024 live sessions, and 512 live rate-limit buckets. Each authentication action permits eight attempts per peer IP and normalized account within a fixed five-minute window. There is no total storage quota.